

MSWLogo and dynamic link libraries

Matjaž Zaversnik, Vladimir Batagelj

*University of Ljubljana, FMF, Department of Mathematics
Jadranska 19, 1000 Ljubljana, Slovenia*

matjaz.zaversnik@fmf.uni-lj.si, vladimir.batagelj@uni-lj.si

Abstract

Dynamic link libraries (DLL) are program modules that contain code, data, or resources that can be shared among many Windows applications. In the paper we will show how to use them to extend MSWLogo functionality and how to write our own DLLs. Some short examples of DLLs usage will be given.

Keywords: MSWLogo, DLL

1. What is a dynamic link library?

A *dynamic link library* (DLL) is a collection of routines that can be called by applications and by other DLLs. DLLs provide a way to modularize applications so that functionality can be updated and reused more easily. They also help reduce memory overhead when several applications use the same functionality at the same time, because although each application gets its own copy of the data, they can share the code. DLLs contain sharable code or resources.

DLLs are separately compiled executables that are linked at runtime to the programs that use them. To distinguish them from standalone executables, files containing compiled DLLs are named with the DLL extension. Programs written in almost any language can call DLLs written in other languages.

In Windows, DLLs are modules that contain functions and data. A DLL is loaded at runtime by its calling modules (EXE or DLL). When a DLL is loaded, it is mapped into the address space of the calling process.

2. DLL support in MSWLogo

MSWLogo provides three commands to work with DLLs.

DLLLOAD *dllname*

This command will load the DLL called *dllname* into MSWLogo. Once loaded you can call functions with `DLLCALL` and once you finished with the DLL or need to use another DLL you can free DLL with `DLLFREE`.

DLLFREE

This command will free the DLL that was loaded with `DLLLOAD`.

status DLLCALL function

This command will call the specified function and will return its output (*status*). The function must be in the DLL that was loaded with `DLLLOAD`. The function is determined by the given list *function*, which must be paired and must contain at least four items. It has the format

```
[typedef name type1 arg1 type2 arg2 type3 arg3 ...]
```

The *type* items specify what the types of function and arguments are, *name* is the name of the function to be called, and *arg* are arguments for the function. Possible types are *v* (void), *w* (word), *l* (dword), *f* (double) and *s* (lpstr). If you do not properly match the argument list, the result will be unpredictable (including crashing Windows). If the argument list is "void" then use [... *v* 0].

This function can handle a wide variety of functions but certainly not all. Since it does allow you to get outside of MSWLogo, you can certainly write your own DLL with a compatible call interface to call more complex interfaces.

3. Using DLL in MSWLogo

Some DLLs are built in the Windows, for example `user32.dll`, `kernel32.dll`, `gdi32.dll`, while many other DLLs are freely available on the internet. To use such a DLL, we have to know which functions are public (exported) and short description of their purpose, parameters and returned value. Using `user32.dll` we can manipulate the windows, menus, buttons, ... To resize the MSWLogo main Window, we should run the following:

```
dllload "user32.dll
make "handle dllcall [l FindWindowA s [MswLogo Screen] l 0]
make "status dllcall ~
  (list "l "MoveWindow "w 1 "l 100 "l 100 "l 0 "l 0 "l :handle)
dllfree
```

First we have to find the handle of the MSWLogo main window. We call the `FindWindowA` function with the name of the window as a parameter. After that we call the `MoveWindow` function to move the window in the upper left corner of the screen and change its width and height to 100 pixels. The last parameter is the repaint flag. Value 1 (true) means that window has to be repainted. Note that ***the parameters must be given in reverse order under Win32***.

This example can be generalized into a function `ResizeWindow` that moves the MSWLogo main window to a position, upper left corner, (*x*,*y*) and resizes it to width *w* and height *h*.

```
to ResizeWindow :x :y :w :h
  dllload "user32.dll
  make "handle dllcall [l FindWindowA s [MswLogo Screen] l 0]
  make "status dllcall ~
    (list "l "MoveWindow "w 1 "l :h "l :w "l :y "l :x "l :handle)
  dllfree
end
```

4. DLL creation

There are several possibilities, how to write your own DLL. We choose Delphi as the most convenient programming environment. To start we have to create new DLL (choose the New option from the File menu, then choose DLL from the New tab). Delphi will generate a new project with a file `Project1.dpr` (you can rename it later). The structure of this file is identical to that of a pascal program, except that a DLL begins with the reserved word `library` (instead of `program`).

The following example shows a Delphi source code for a DLL with two exported functions, `Min` and `Max`.

```
library MinMax;

function min(x, y: integer): integer; stdcall;
begin
    if x < y then result := x else result := y;
end;

function max(x, y: integer): integer; stdcall;
begin
    if x > y then result := x else result := y;
end;

exports
    min, max;

begin
end.
```

DLLs can define two kinds of functions: exported and internal. The exported functions can be called by other modules. Internal functions can only be called from within the DLL where they are defined. To make some functions public (exported) we have to list their names in an `exports` clause.

If we want our DLL to be available to applications written in other languages, it's safest to specify `stdcall` in the declarations of exported functions. Other languages may not support Object Pascal's default register calling convention.

As we saw, there are several types of parameters, supported by MSWLogo. The void type (letter `v`) is used only, when the function is without parameters or when it returns no result. There are two different integer types: `word` (letter `w`) (16-bit integer) and `dword` (letter `l`) (32-bit integer). The corresponding pascal types are `smallint` or `word` (signed and unsigned 16-bit integer) and `integer` or `cardinal` (signed and unsigned 32-bit integer). MSWLogo supports only one type (letter `f`) of real numbers. The corresponding pascal type is `double` (64-bit real number). To pass a string (letter `s`) we should use `PChar` (pointer to character) type in Delphi. If the function result is a string, MSWLogo gets it in a list, so we have to use command `first` to get the result out of the list.

5. Examples

5.1. Text analysis

Text analysis is an example of simple DLL, which contains only one exported and several internal functions. Exported function gets several parameters: two filenames and some integer flags. It counts, how many times different words occur on the first file and writes the results on the second file. For each word from the input file we have a line in the output file, consisting from the frequency and the word itself. The prototype of exported function is:

```
procedure Count(  
    input, output: PChar;  
    sort, order, tail, delimiter, match, ignore: word  
);
```

The meanings of the parameters are:

input	=	name of the input file
output	=	name of the output file
sort	=	how to sort the lines in the output file 0 = sort alphabetically 1 = sort by frequencies then alphabetically in the same order 2 = sort by frequencies then alphabetically in the reverse order
order	=	sort order 0 = ascending 1 = descending
tail	=	how to sort the words 0 = by their beginnings 1 = by their tails
delimiter	=	what are the words 0 = words are delimited by white spaces and punctuations 1 = words are delimited by white spaces only
match	=	the matching of lower and upper case letters 0 = do not match case 1 = match case
ignore	=	ignore all words shorter than value of this parameter

We also provide logo command (`WordCount.lgo`) which makes the use of the described function easier.

```
to WordCount :input :output :sort :order :tail :delimiter :match :ignore  
    dllload "WordCount.dll  
    dllcall (list  
        "v "Count "w :ignore "w :match "w :delimiter "w :tail  
        "w :order "w :sort "s :output "s :input  
    )  
    dllfree  
end
```

The following function returns a list of pairs (frequency, word) from a given file with the frequency greater or equal to a given number, sorted in descending order of their frequencies.

```
to frequent :input :freq
  WordCount :input "temp.txt 1 1 0 0 0 0
  openread "temp.txt setread "temp.txt
  make "words [] make "c :freq
  while [not or eofp (:c < :freq)] [
    make "line rl make "c first :line
    make "words lput :line :words
  ]
  setread [] close "temp.txt
  output ifelse empty? :words [[]] [bl :words]
end
```

To demonstrate the use of this function we choose at the Project Gutenberg site the text of Hamlet written by William Shakespeare. The request `frequent "hamlet.txt 50` gives the answer:

```
[
  [993 the]      [863 and]      [685 to]        [610 of]        [574 i]         [527 you]
  [511 a]        [502 my]       [419 it]        [400 in]        [377 that]      [337 ham]
  [328 is]       [300 not]     [285 his]       [276 this]     [254 with]      [253 your]
  [249 but]      [245 for]     [228 me]        [223 d]        [211 what]      [211 lord]
  [205 as]       [202 he]      [192 be]        [189 so]       [178 him]       [178 haue]
  [172 king]     [149 will]    [137 no]        [131 on]       [130 our]       [128 we]
  [124 s]        [123 are]     [114 if]        [109 all]      [108 then]      [107 thou]
  [107 shall]    [106 by]     [104 let]       [104 do]       [104 come]      [103 or]
  [100 hamlet]   [ 98 good]    [ 95 hor]       [ 95 her]      [ 92 there]     [ 92 now]
  [ 91 t]        [ 90 thy]     [ 90 they]      [ 89 more]     [ 89 how]       [ 87 from]
  [ 85 enter]    [ 84 was]     [ 81 oh]        [ 81 at]       [ 80 like]      [ 77 most]
  [ 73 would]    [ 71 well]    [ 71 know]      [ 69 tis]      [ 68 selfe]     [ 67 them]
  [ 67 o]        [ 66 loue]    [ 65 may]       [ 64 vs]       [ 63 did]       [ 62 th]
  [ 62 sir]      [ 62 qu]     [ 60 why]       [ 60 which]    [ 60 too]       [ 60 laer]
  [ 59 giue]     [ 58 thee]    [ 58 must]      [ 58 ile]      [ 57 hath]      [ 56 where]
  [ 56 should]   [ 56 ophe]    [ 55 speake]    [ 55 out]      [ 54 make]      [ 53 an]
  [ 52 vpon]     [ 52 such]    [ 52 say]       [ 52 am]       [ 51 when]      [ 51 doe]
  [ 50 heere]    [ 50 father]
]
```

5.2. Long arithmetics

This DLL contains several functions to make basic arithmetic operations with very big integers represented as strings. These functions are:

```
function LongSum(a, b: PChar): PChar;
function LongSub(a, b: PChar): PChar;
function LongProd(a, b: PChar): PChar;
function LongDiv(a, b: PChar): PChar;
function LongMod(a, b: PChar): PChar;
function LongPower(a: PChar; n: word): PChar;
```

To multiply two big numbers, say a and b, we have to run the following:

```
dllload "LongInt.dll
make "result first dllcall (list "s "LongProd "s :b "s :a)
dllfree
```

To make the use of this DLL as simple as possible we provide logo commands with the same names as the corresponding pascal functions. So we have to load these commands (`longint.lgo`) and use them as it is shown bellow.

```
make "result LongProd :a :b
```

Using these Logo commands it is very simple to calculate famous numbers (e , π) to specified number of digits exactly. Here is the command `EDigits`, which computes first n digits of number e .

```
to EDigits :n
  localmake "b LongPower "10 :n + 3
  localmake "s :b
  localmake "c :b
  localmake "k 0
  while [not (:c = "0)] [
    make "k :k + 1
    make "c LongDiv :c (word :k)
    make "s LongSum :s :c
  ]
  make "s LongSum :s "500
  output (word first :s ". bf bl bl bl :s)
end
```

The request `EDigits 100` gives the answer:

```
2.71828182845904523536028747135266249775724709369995
95749669676277240766303535475945713821785251664274
```

6. Conclusions

Although we found DLLs very usefull, we have to consider some limitations of their use in MSWLogo. We can use at most one DLL at the same time. If we have loaded a DLL already and we need functionality of another DLL, we have to free the first DLL and load the second one. Another limitation is the length of string parameters. Any string (parameter or returned value) can be of length at most 255. It is also very difficult (read impossible) to pass a complicated structure as parameter to the function in DLL.

The complete source code of given examples can be found at

<http://www.educa.fmf.uni-lj.si/Logo/EuroLogo.01/>

7. References

Mills G (1998) *MSWLogo Reference Manual*

<http://www.softronix.com/logo.html>

Pacheco X, Teixeira S (1996) *Delphi 2 Developer's Guide*, second edition, Borland Press

Project Gutenberg

<http://promo.net/pg/>